

DOCUMENTACIÓN DE IMPLEMENTACIÓN

Proyecto 1 - Captura la Bandera Multijugador

Alcance del documento

Documentación formal del proceso de desarrollo desde la versión 1 hasta la versión 9, correspondiente al estado final de entrega. Incluye arquitectura, protocolo de comunicación, reglas del juego, pruebas, cronología en Git, uso de inteligencia artificial, estado de cumplimiento y la experiencia personal registrada en cada etapa.

Campo	Información
Estudiante	Emily Urbina
Carné	23001552
Curso	Ciencias de la Computación 8 - 2026
Motor y lenguaje	Godot Engine 4.7 Stable / GDScript
Tipo de entorno	Aplicación 3D en primera persona
Repositorio	Project1_CC8_2026
Versión documentada	Versión 9
Fecha de actualización	28 de julio de 2026

Universidad Galileo

Guatemala, 2026

Tabla de contenidos

1. Resumen ejecutivo
2. Marco documental: instrucciones del proyecto y protocolo técnico
3. Requisitos del proyecto y estado de cumplimiento
4. Descripción general y alcance
5. Tecnologías, estructura y arquitectura
6. Implementación de la comunicación
7. Protocolo de mensajes y framing
8. Reglas, autoridad y sincronización
9. Manejo de errores, desconexiones y seguridad
10. Proceso de desarrollo por versiones
11. Pruebas realizadas y evidencia
12. Uso de Git e inteligencia artificial
13. Ejecución del proyecto
14. Estado actual, limitaciones y trabajo pendiente
15. Conclusiones

Anexos

Nota de actualización

El documento refleja el estado funcional alcanzado hasta la versión 9, entrega final del proyecto. La representación visual de jugadores remotos y la vista gráfica del servidor ya se implementaron (versión 8); el único apartado que permanece pendiente por naturaleza propia del entregable es la prueba de interoperabilidad con los proyectos heterogéneos de toda la clase, que solo puede completarse el día de la prueba conjunta.

1. Resumen ejecutivo

El proyecto consiste en una implementación multijugador de Captura la Bandera desarrollada de forma individual en Godot Engine 4.7 mediante GDScript. La aplicación se diseñó para operar como cliente y como servidor, utilizando sockets básicos y un protocolo común acordado por la clase. El objetivo técnico principal es garantizar que cada proyecto pueda descubrir servidores, establecer una conexión y participar en partidas interoperables sin depender del lenguaje o motor utilizado por los demás estudiantes.

La arquitectura emplea UDP exclusivamente para el descubrimiento de servidores y TCP para toda la comunicación de la partida. Los mensajes se representan como objetos JSON codificados en UTF-8. En TCP, cada mensaje se delimita con un salto de línea y se procesa mediante buffers capaces de manejar mensajes fragmentados o varios mensajes recibidos en una sola lectura.

El servidor conserva la autoridad sobre las posiciones, el movimiento, la bandera, las fases de la partida y la victoria. El cliente únicamente envía intenciones de movimiento e interacción, y representa el estado oficial recibido. Esta separación evita que un cliente pueda declarar una posición o una victoria por cuenta propia.

Hasta la versión 9, entrega final, se implementó el escenario 3D, el jugador en primera persona, la comunicación TCP, el descubrimiento UDP, el ciclo completo de partida, la adaptación al protocolo vigente, las pruebas remotas mediante VPN, una interfaz gráfica para buscar, seleccionar y conectar a un servidor sin argumentos de terminal, la representación visual de todos los jugadores remotos dentro de cada cliente, una vista gráfica propia para el modo servidor, la corrección del descubrimiento UDP para redes VPN con múltiples interfaces, un conteo regresivo visible en pantalla y un menú de pausa.

2. Marco documental: instrucciones del proyecto y protocolo técnico

La documentación se apoya en dos fuentes distintas y complementarias. El documento "Proyecto 1 - Captura la Bandera Multijugador" contiene las instrucciones académicas, las reglas generales del juego, las limitaciones de implementación y los elementos obligatorios del entregable. El documento "Estándar Protocolo CTF - CC8 2026" define, por separado, la forma exacta en que deben comunicarse todos los proyectos para lograr interoperabilidad.

Documento	Función dentro del proyecto	Contenido utilizado
Instrucciones del proyecto	Define qué debe entregarse y qué condiciones académicas debe cumplir la solución.	Juego funcional; modos servidor y cliente; hasta 100 conexiones; visualización de jugadores; documentación desde versión 1; Git; IA y prompts; interoperabilidad con toda la clase.
Estándar de protocolo CTF	Define cómo deben comunicarse cliente y servidor de forma compatible.	UDP para descubrimiento; TCP para juego; JSON UTF-8; framing por salto de línea; mensajes; estados; errores; constantes; reglas autoritativas y desconexiones.

Criterio de interpretación

Las instrucciones del proyecto se utilizan como lista principal de cumplimiento. El protocolo se utiliza como especificación técnica de implementación. Un requisito del entregable no debe confundirse con

un campo o regla del protocolo, y una regla del protocolo no sustituye una exigencia académica del proyecto.

3. Requisitos del proyecto y estado de cumplimiento

De acuerdo con el documento de instrucciones del proyecto, se exige un juego funcional que pueda trabajar como servidor o cliente, soporte múltiples conexiones, descubra servidores, valide las reglas en el servidor, muestre los jugadores conectados y documente todo el proceso desde la primera versión, incluyendo Git y el uso de inteligencia artificial.

Requisito explícito	Implementación actual	Estado
Juego funcional de Captura la Bandera	Movimiento, captura, robo, victoria, fin de ronda y reinicio del ciclo.	Implementado
Modo servidor y modo cliente	Escena de servidor independiente y cliente integrado en GameWorld.	Implementado
Descubrimiento del servidor	Broadcast UDP, búsqueda unicast por IP y conexión TCP directa.	Implementado
Selección del servidor	Interfaz ServerBrowser con lista seleccionable.	Implementado
Soporte multijugador	Servidor preparado para hasta 100 conexiones y probado con dos clientes.	Parcialmente verificado
Servidor autoritativo	Calcula posiciones, valida interacción, bandera y victoria.	Implementado
Countdown antes de iniciar	Cuenta regresiva de 5 a 1 con mínimo de dos jugadores.	Implementado
Todos los clientes muestran a todos los jugadores	Desde la versión 8, cada cliente instancia, mueve y elimina las cápsulas de los demás jugadores según el arreglo <code>players[]</code> de cada state, con etiqueta de nombre e interpolación de posición.	Implementado
Servidor muestra a todos los jugadores	Desde la versión 8, <code>server_player_view.tscn</code> muestra el mapa, el círculo, la bandera y a todos los jugadores conectados en tiempo real, con cámara cenital y HUD de fase, puerto y cantidad de jugadores.	Implementado
Interoperabilidad con toda la clase	Protocolo adaptado y probado por VPN entre dos computadoras propias y con una compañera de clase (versión 8); falta la prueba masiva formal con todos los proyectos heterogéneos el día acordado.	Pendiente de validación final
Cronología en Git	Commits separados por versiones y bitácora asociada.	Implementado
Documentación desde versión 1	Versiones 1 a 9 descritas en este documento.	Implementado
IA y prompts utilizados	Registro separado en <code>docs/prompts_ia.md</code> y referencias en la documentación.	Implementado

Lectura del estado

“Implementado” significa que la función existe y fue probada en el entorno descrito. “Parcialmente verificado” indica que la estructura está preparada, pero no se ha demostrado todavía con 100 clientes. “Pendiente” identifica requisitos del entregable que todavía requieren desarrollo antes de la entrega final.

3. Descripción general y alcance

3.1 Objetivo del juego

1. El jugador aparece fuera del círculo central.
2. Debe entrar al círculo y acercarse a la bandera ubicada en el centro.
3. Presiona la tecla de interacción para tomar la bandera cuando se encuentra dentro del radio permitido.
4. Otro jugador puede robar la bandera si se acerca al portador y también interactúa.
5. El portador gana únicamente cuando realiza la transición de estar dentro o en el borde del círculo a estar completamente fuera mientras conserva la bandera.

3.2 Alcance técnico

- Entorno gráfico 3D en primera persona.
- Mapa lógico de 1000 x 1000 unidades, representado en Godot con escala 1:10.
- Cliente y servidor desarrollados con sockets integrados de Godot.
- Descubrimiento de servidores por UDP en el puerto fijo 8888.
- Comunicación de partida por TCP en el puerto anunciado por cada servidor; para este proyecto se utiliza 8889.
- Formato de mensajes JSON codificado en UTF-8.
- Modelo servidor-autoritativo para preservar la integridad de la partida.
- Interfaz gráfica para búsqueda, selección y conexión a servidores.

3.3 Fuera del alcance actual

- Cifrado TLS/DTLS y autenticación de usuarios.
- Migración automática de servidor.
- Reconexión a una partida que ya está en curso.
- Colisión física entre cuerpos de jugadores.
- Sistema de cuentas, puntuación persistente o base de datos.

4. Tecnologías, estructura y arquitectura

4.1 Tecnologías utilizadas

Elemento	Tecnología / decisión	Justificación
Motor	Godot Engine 4.7 Stable	Permite crear el entorno 3D y ofrece sockets básicos sin dependencias externas.

Elemento	Tecnología / decisión	Justificación
Lenguaje	GDScript	Integración directa con nodos, escenas, señales y ciclo de ejecución de Godot.
Transporte	UDP + TCP	UDP facilita descubrimiento; TCP garantiza orden y entrega para la partida.
Serialización	JSON UTF-8	Formato común, legible e interoperable entre lenguajes.
Control de versiones	Git y GitHub Desktop	Registra cronología, cambios funcionales y evidencia del desarrollo.
Pruebas remotas	ZeroTier y Hostspots	Permite simular una red privada entre computadoras en ubicaciones distintas.

4.2 Organización del proyecto

```

Project1_CC8_2026/
├── scenes/
│   ├── levels/game_world.tscn
│   ├── network/server.tscn
│   ├── network/server_player_view.tscn
│   ├── player/player.tscn
│   ├── player/remote_player.tscn
│   ├── ui/server_browser.tscn
│   └── ui/pause_menu.tscn
├── scripts/
│   ├── network/game_server.gd
│   ├── network/game_client.gd
│   ├── network/server_discovery.gd
│   ├── player/player.gd
│   ├── player/remote_player.gd
│   ├── ui/server_browser.gd
│   └── ui/pause_menu.gd
├── docs/
│   ├── 01_bitacora.md
│   ├── prompts_ia.md
│   └── prompts_ia/*.png
└── tests/evidence/*.png

```

4.3 Escena principal

La escena GameWorld contiene la geometría del nivel, el círculo central, la bandera, la iluminación, el ambiente, el jugador local, el cliente de red, el componente de descubrimiento y un CanvasLayer que muestra la interfaz de búsqueda de servidores.

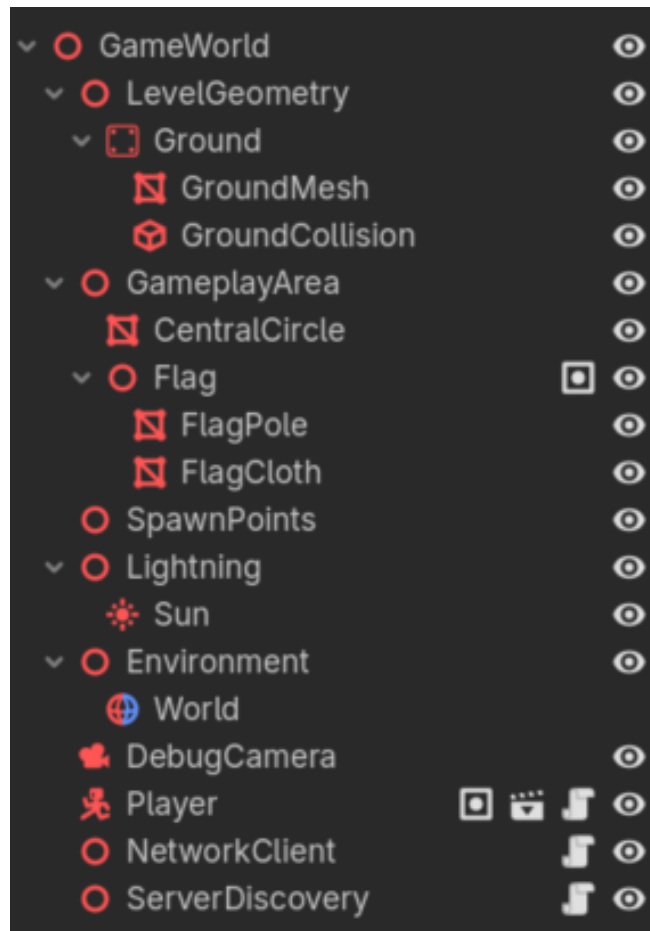


Figura 1. Árbol de nodos de la escena GameWorld durante la integración de la interfaz.

4.4 Responsabilidades por componente

Componente	Responsabilidad principal
game_server.gd	Aceptar conexiones, validar mensajes, administrar fases, simular movimiento, controlar la bandera, determinar victoria y emitir estados. Desde la versión 8, expone además sus diccionarios internos players y flag a server_player_view.tscn para la vista gráfica del servidor.
game_client.gd	Conectarse por TCP, enviar join/input/interact, procesar mensajes, actualizar el jugador local, representar la bandera (reparentada a cualquier portador, local o remoto) y, desde la versión 8, crear/mover/eliminar las cápsulas de RemotePlayer según el arreglo players[] recibido en cada state; muestra además el conteo regresivo en pantalla.
server_discovery.gd	Enviar discover, recibir server_info, validar respuestas y entregar la lista de servidores. Desde la versión 8 detecta automáticamente todas las direcciones IPv4 locales (incluidos adaptadores VPN) para calcular el broadcast de cada subred y filtra direcciones link-local; desde la versión 9 descarta además las respuestas que provienen de la propia máquina.
player.gd	Leer teclado y mouse, controlar cámara y enviar intenciones al cliente de red.
server_browser.gd	Presentar servidores, permitir selección, búsqueda por IP y conexión directa.

Componente	Responsabilidad principal
server.tscn	Ejecutar el servidor como proceso independiente.
game_world.tscn	Ejecutar el cliente y mostrar el entorno 3D.

5. Implementación de la comunicación

5.1 Arquitectura híbrida

La solución separa el descubrimiento de la comunicación de juego. UDP se utiliza solo para localizar servidores; una vez elegido un servidor, el cliente abre una única conexión TCP y la mantiene durante el lobby, countdown, partida, pausa posterior y rondas siguientes.

```

Cliente -- UDP discover --> 255.255.255.255:8888 / broadcast de subred
Servidor -- UDP server_info --> IP y puerto efímero del cliente
Cliente -- TCP connect --> IP_servidor:tcp_port
Cliente -- TCP join --> Servidor
Servidor -- TCP welcome/lobby/countdown/start/state/game_over --> Cliente

```

5.2 Descubrimiento automático

- El cliente abre un socket UDP temporal y habilita broadcast.
- Envía {"type":"discover","v":1} de forma periódica durante la ventana de búsqueda.
- El servidor escucha permanentemente en UDP 8888.
- Ante un discover válido de versión 1, responde por unicast con server_info.
- El cliente toma la IP real del servidor desde la dirección de origen del datagrama recibido.
- Cada resultado válido se agrega a una lista para que el usuario seleccione el servidor.
- Desde la versión 8, el servidor detecta automáticamente todas sus direcciones IPv4 locales (incluyendo adaptadores virtuales como el de ZeroTier) y calcula el broadcast de cada subred, en lugar de depender de un parámetro manual de broadcast; también descarta direcciones link-local (169.254.x.x) para evitar envíos fallidos sin efecto.
- Desde la versión 9, el cliente descarta las respuestas server_info cuya IP de origen coincida con una de las direcciones IPv4 de la propia máquina, evitando ver el propio servidor repetido por cada interfaz de red activa.

5.3 Respaldos de conexión

Vía	Uso	Ventaja
Broadcast UDP	Búsqueda automática sin conocer la IP.	Experiencia ideal dentro de una LAN compatible.
UDP unicast por IP	El usuario escribe la IP; se conserva discover/server_info.	Útil en VPN o redes que bloquean broadcast.
TCP directo IP:puerto	Conexión al puerto conocido sin descubrimiento.	Último respaldo para diagnosticar conectividad.

5.4 Interfaz de selección

La escena `ServerBrowser` es una interfaz de tipo `User Interface` instanciada dentro de un `CanvasLayer`. Al iniciar el cliente sin argumentos, el juego espera la acción del usuario. El botón de búsqueda solicita al componente `ServerDiscovery` que busque servidores; al finalizar, los resultados se muestran en un `ItemList`. El usuario selecciona una fila y presiona `Conectar`. La interfaz aplica el nombre elegido y solicita a `NetworkClient` abrir la conexión TCP.

- Nombre del jugador.
- Botón para búsqueda automática.
- Estado de la búsqueda.
- Lista de servidores encontrados.
- Botón de conexión al servidor seleccionado.
- Campo de IP y botón de búsqueda unicast.
- Campo de puerto y botón de conexión TCP directa.

Cuando el cliente recibe `welcome`, la interfaz se oculta y el mouse vuelve al modo capturado para controlar la cámara. Este comportamiento confirma que la conexión y el join fueron aceptados por el servidor.

5.5 Conteo regresivo visual y menú de pausa

Desde la versión 9, `game_client.gd` muestra en pantalla el número recibido en cada mensaje `countdown` mediante un `Label` centrado, ocultándolo automáticamente al iniciar la partida, al volver al lobby, al terminar la ronda o al perder la conexión. Además, la tecla `ESC` abre un menú de pausa (`pause_menu.tscn / pause_menu.gd`) que libera el mouse durante la partida y ofrece dos opciones: `Reanudar`, que cierra el menú y vuelve a capturar el mouse sin afectar la conexión, y `Salir`, que cierra el programa por completo. La opción de volver al buscador de servidores sin cerrar el programa se descartó tras detectar un error donde la bandera desaparecía al reconectarse a una nueva partida; por falta de tiempo para corregir la causa raíz antes de la entrega, se simplificó a un cierre completo de la aplicación.

6. Protocolo de mensajes y framing

6.1 Formato común

- Todos los mensajes son objetos `JSON`.
- La codificación es `UTF-8`.
- Todo mensaje contiene el campo `type`.
- El campo `v` aparece únicamente donde el protocolo lo define y debe ser 1.
- Los valores numéricos se envían como números, no como cadenas.
- Los campos desconocidos deben ignorarse para permitir extensibilidad.
- El tamaño máximo de un mensaje es 64 KB.

6.2 Framing en TCP

TCP entrega un flujo continuo de bytes; no conserva los límites de los mensajes. Por esta razón, cada `JSON` se envía en una sola línea y termina con el carácter `\n`. Tanto cliente como servidor conservan un buffer por conexión.

6. Agregar al buffer los bytes recién recibidos y convertirlos desde `UTF-8`.
7. Buscar el siguiente salto de línea.
8. Extraer el texto anterior al salto como un mensaje completo.
9. Eliminar un retorno de carro final si se recibió `\r\n`.

10. Parsear el JSON y procesarlo.
11. Conservar en el buffer el fragmento restante para la siguiente lectura.
12. Repetir mientras existan mensajes completos.

```
[JSON completo 1]\n[JSON completo 2]\n[fragmento incompleto del JSON 3...]
```

6.3 Catálogo de mensajes

Mensaje	Dirección	Fase	Función
discover	Cliente -> UDP	Cualquiera	Buscar servidores.
server_info	Servidor -> UDP	Cualquiera	Anunciar nombre, puerto, estado y jugadores.
join	Cliente -> Servidor	Lobby	Solicitar ingreso con versión y nombre.
input	Cliente -> Servidor	Playing	Enviar dirección de movimiento.
interact	Cliente -> Servidor	Playing	Intentar capturar o robar la bandera.
welcome	Servidor -> Cliente	Lobby	Asignar player_id y configuración fija.
lobby	Servidor -> Cliente	Lobby	Actualizar jugadores y señalar regreso al lobby.
countdown	Servidor -> Cliente	Countdown	Informar segundos restantes.
start	Servidor -> Cliente	Cambio a Playing	Marcar inicio exacto de la partida.
state	Servidor -> Cliente	Playing	Replicar posiciones y bandera.
game_over	Servidor -> Cliente	Fin de partida	Anunciar ganador.
error	Servidor -> Cliente	Cualquiera	Comunicar rechazo y motivo normativo.

6.4 Constantes y límites

Constante	Valor	Aplicación
map_size	1000	Mapa lógico de 1000 x 1000.
circle_radius	300	Radio lógico del círculo.
player_radius	15	Radio del jugador y margen de límites.
interact_radius	40	Distancia máxima de captura o robo.
speed	200	Velocidad lógica por segundo.
tick_rate	20	Estados por segundo.
countdown_seconds	5	Cuenta regresiva 5, 4, 3, 2, 1.
min_players	2	Cantidad mínima para iniciar.
post_game_seconds	5	Pausa antes de volver al lobby.

Constante	Valor	Aplicación
spawn_radius	350 a 450	Anillo aleatorio fuera del círculo.
victory_distance	315	circle_radius + player_radius.
max_players	100	Máximo de conexiones.
name_max_length	20	Máximo de caracteres del nombre.
message_max_size	64 KB	Máximo por mensaje.

7. Reglas, autoridad y sincronización

7.1 Ciclo de vida

Lobby -> Countdown (5 s) -> Playing -> Game Over -> Pausa (5 s) -> Lobby

Cuando termina la pausa posterior, las conexiones TCP permanecen abiertas. El servidor vuelve a lobby y envía la lista actualizada. Si continúan conectados al menos dos jugadores, puede iniciar inmediatamente otra cuenta regresiva; por eso una nueva partida comienza sin que los clientes se reconecten.

7.2 Movimiento y coordenadas

- El protocolo usa origen (0,0) en la esquina superior izquierda.
- X aumenta hacia la derecha y Y aumenta hacia abajo.
- Godot usa X-Z para el plano del suelo y Y para altura.
- La conversión visual resta 500 y divide entre 10.
- El cliente envía dir.x y dir.y con valores -1, 0 o 1.
- El servidor normaliza diagonales, integra velocidad y restringe la posición al intervalo [15,985].

```
Godot.x = (protocolo.x - 500) / 10
Godot.z = (protocolo.y - 500) / 10
```

7.3 Spawn

Al comenzar la partida, el servidor asigna una posición aleatoria uniforme en un anillo entre 350 y 450 unidades alrededor de (500,500). Esto garantiza que todos aparezcan fuera de la distancia de victoria.

7.4 Captura, robo y bandera

- La bandera libre se encuentra en (500,500) y owner es null.
- Un interact dentro de 40 unidades captura la bandera si está libre.
- Un interact dentro de 40 unidades del portador roba la bandera.
- No existe tiempo de espera ni inmunidad.
- Mientras está portada, la posición de la bandera coincide con la posición del portador.
- Si el portador se desconecta, la bandera regresa al centro.

7.5 Victoria con transición obligatoria

No basta con poseer la bandera fuera del círculo. El servidor registra si el portador estuvo dentro o en el borde, con distancia menor o igual a 315, y únicamente declara victoria cuando después pasa a una distancia estrictamente mayor que 315 sin perder la bandera. Quien roba la bandera estando ya fuera debe volver a entrar y salir para ganar.

7.6 Autoridad del servidor

Acción	Cliente envía	Servidor decide
Movimiento	Dirección	Posición, velocidad, normalización y límites.
Captura	interact	Distancia y disponibilidad de la bandera.
Robo	interact	Distancia al portador y cambio de owner.
Victoria	Nada especial	Transición dentro -> fuera conservando bandera.
Fase	Acción solicitada	Aceptar o rechazar según lobby/countdown/playing.

8. Manejo de errores, desconexiones y seguridad

8.1 Validación de mensajes

El servidor trata toda entrada como no confiable. Antes de cambiar el estado verifica JSON, type, campos requeridos, tipos, versión, fase, nombre y rango de direcciones. Un mensaje inválido no debe alterar la partida.

Código	Causa	Cierre
INVALID_JSON	Texto no parseable como JSON.	No; tras repetición puede cerrarse.
UNKNOWN_TYPE	Mensaje no registrado.	No
MISSING_FIELD	Falta campo obligatorio.	No
INVALID_FIELD	Tipo o valor incorrecto.	No
INVALID_PHASE	Acción fuera de fase.	No
VERSION_MISMATCH	v distinta de 1.	Sí
LOBBY_FULL	Se alcanzó el máximo.	Sí
NAME_INVALID	Nombre vacío, largo o con controles.	No
GAME_STARTED	join durante countdown/playing.	Sí
MESSAGE_TOO_LARGE	Mensaje superior a 64 KB.	Sí
NOT_JOINED	Acción antes de join.	No

8.2 Desconexiones

- En lobby: se elimina al jugador y se difunde la lista actualizada.
- En countdown: si quedan menos de dos jugadores, se aborta y se vuelve al lobby.
- En partida: se elimina al jugador del estado; la partida continúa.
- Si se desconecta el portador: la bandera regresa al centro.
- Si se desconectan todos: se reinicia el ciclo en lobby.
- Si se desconecta el servidor: los clientes detienen la partida.
- La versión 1 no define timeout de inactividad; las conexiones rotas se detectan por TCP.

8.3 Consideraciones de seguridad

El protocolo no incluye cifrado ni autenticación y se supone ejecutado en una LAN o entorno controlado. Para reducir abuso, el servidor limita conexiones, nombres y tamaño de mensajes, valida tipos y no acepta coordenadas del cliente. Durante pruebas remotas se utilizó una VPN privada para aislar el tráfico entre dispositivos autorizados.

9. Proceso de desarrollo por versiones

Las versiones se definieron como incrementos funcionales identificables. Cada una cuenta con un commit, evidencia y registro en la bitácora.

Versión 1. Configuración inicial

Objetivo: Preparar el repositorio, el proyecto y una estructura organizada antes de implementar el juego.

Implementación principal

- Se creó el repositorio Project1_CC8_2026.
- Se inicializó Godot 4.7 Stable con GDScript.
- Se configuró .gitignore para archivos generados por Godot.
- Se organizaron carpetas de escenas, scripts, documentación y pruebas.
- Se inició 01_bitacora.md para conservar la cronología.

Pruebas y resultado

- El proyecto abrió correctamente desde el repositorio.
- Git reconoció únicamente los archivos relevantes.
- La estructura permitió separar código, escenas y evidencia desde el inicio.

Commit de referencia: chore: configurar estructura inicial del proyecto Godot

Hash del commit: 2641f25 Fecha: 18 de julio de 2026

Experiencia personal de la versión

Reflexión personal

Al comenzar, mi prioridad fue preparar una estructura que evitara desorden conforme creciera el proyecto. Esta etapa me permitió familiarizarme con la relación entre Godot y Git y entender qué archivos debían versionarse. El principal aprendizaje fue que una organización inicial clara facilita documentar y probar cada avance.

Versión 2. Primera versión visual 3D

Objetivo: Construir un greybox del mapa y adaptar las medidas lógicas del protocolo a un entorno 3D.

Implementación principal

- Se creó GameWorld con suelo, colisión, iluminación y ambiente.
- Se agregó el círculo central y una bandera provisional.
- Se estableció la escala 1:10: 1000 unidades lógicas equivalen a 100 unidades de Godot.
- Se reservó el eje Y para altura y se utilizó X-Z para el plano.

Pruebas y resultado

- El escenario se renderizó correctamente.
- El suelo y la geometría coincidieron con las medidas previstas.
- La bandera quedó ubicada en el centro visual del mapa.

Evidencia asociada: evidence/greybox_v0.2.0.png

Commit de referencia: feat: crear primera versión visual 3D del escenario

Hash del commit: d0cc4bd Fecha: 19 de julio de 2026

Experiencia personal de la versión

Reflexión personal

Esta versión me ayudó a convertir un protocolo abstracto en un espacio visual. El mayor reto fue entender que las coordenadas del documento no podían copiarse directamente a Godot y que necesitaba una conversión consistente. La escala 1:10 permitió trabajar con dimensiones manejables sin perder equivalencia lógica.

Versión 3. Jugador y reglas locales

Objetivo: Crear un jugador controlable y validar localmente movimiento, interacción y victoria antes de introducir la red.

Implementación principal

- Se creó Player con CharacterBody3D, colisión y cámara.
- Se implementó movimiento WASD, gravedad y colisión con el suelo.
- Se configuró cámara en primera persona y captura del mouse.
- Se aplicaron límites físicos del mapa.
- Se agregó interacción con E, seguimiento de bandera y victoria local provisional.

Decisiones y cambios de enfoque

- Se sustituyó la idea de una cámara de tercera persona por una cámara de primera persona porque la orientación y el control resultaron más claros.

Pruebas y resultado

- Movimiento y cámara funcionaron con teclado y mouse.
- El jugador permaneció dentro del mapa.
- La bandera solo se capturó dentro de la distancia permitida.

- La condición local de victoria se activó al salir completamente del círculo.

Evidencia asociada: tests/evidence/prototipo_funcional_v0.3.0.png

Commit de referencia: feat: implementar jugador y reglas locales de prueba

Hash del commit: 6b35c5b Fecha: 21 de julio de 2026

Experiencia personal de la versión

Reflexión personal

Primero necesité comprobar las mecánicas sin agregar la complejidad de la red. Esto facilitó identificar errores de cámara, escala y límites. La prueba local no se consideró la solución final, sino una forma de aprender el motor y verificar que las reglas podían representarse correctamente.

Versión 4. Primera implementación TCP

Objetivo: Trasladar las reglas principales a una arquitectura cliente-servidor y comprobar la conexión con un cliente.

Implementación principal

- Se creó server.tscn y game_server.gd.
- Se agregó NetworkClient y game_client.gd a GameWorld.
- Se utilizó TCP 8889 para la partida.
- Se implementaron JSON UTF-8, delimitación por \n y buffers por conexión.
- Se implementaron join, input, interact y respuestas welcome, lobby, countdown, start, state, game_over y error.
- El servidor pasó a calcular la posición oficial y las reglas.

Pruebas y resultado

- Servidor y cliente se conectaron en 127.0.0.1.
- El servidor asignó un player_id y envió la configuración.
- El cliente recibió countdown y start.
- El movimiento se habilitó únicamente después de start.

Evidencia asociada: tests/evidence/prueba_conexion_tcp_1.png

Commit de referencia: feat: implementar primera conexion TCP cliente-servidor

Hash del commit: 0e48275 Fecha: 21 de julio de 2026

Experiencia personal de la versión

Reflexión personal

Esta fue la transición más importante: las reglas dejaron de depender del cliente. Aprendí que TCP no entrega mensajes completos automáticamente y que era necesario implementar un buffer. También comprendí la diferencia entre enviar una intención de movimiento y permitir que el cliente establezca su propia posición.

Versión 5. Descubrimiento UDP y dos clientes

Objetivo: Completar la conexión mediante descubrimiento y comprobar que dos clientes puedan iniciar una partida.

Implementación principal

- Se creó `server_discovery.gd`.
- Se abrió UDP 8888 en el servidor.
- Se implementaron `discover` y `server_info`.
- Se mantuvo TCP para toda la partida.
- Se agregó búsqueda por broadcast y conexión manual de respaldo.
- Se configuró countdown al alcanzar dos jugadores.

Pruebas y resultado

- Un cliente encontró el servidor mediante UDP.
- Dos instancias se conectaron al mismo servidor.
- El countdown comenzó cuando se unió el segundo jugador.
- Ambos clientes recibieron start.

Evidencia asociada: `tests/evidence/prueba_dos_conexiones.png`

Commit de referencia: feat: implementar descubrimiento UDP y conexión de dos clientes

Hash del commit: 90fe981 Fecha: 23 de julio de 2026

Experiencia personal de la versión

Reflexión personal

Con esta etapa el proyecto dejó de depender únicamente de 127.0.0.1. La separación entre UDP y TCP me permitió comprender que descubrir un servidor y jugar son procesos distintos. La prueba con dos clientes confirmó que el servidor podía coordinar una fase común para más de una conexión.

Versión 6. Adaptación al protocolo actualizado y VPN

Objetivo: Alinear la implementación con la especificación vigente y comprobar la comunicación entre dos computadoras en redes distintas.

Implementación principal

- Se adoptó la versión 1.2.0 del documento y la versión de cable v=1.
- Se reforzaron validaciones de versión, nombre, campos, tipos y tamaño.
- Se implementó el ciclo lobby-countdown-playing-game over-lobby.
- Se ajustaron spawn, captura, robo y transición obligatoria de victoria.
- Se agregaron broadcast, discover unicast y TCP directo como vías de conexión.
- Se mantuvo al servidor como autoridad completa.

Pruebas y resultado

- Se creó una red privada con ZeroTier.
- Una segunda computadora envió `discover` por UDP unicast a la IP virtual del servidor.
- El cliente recibió `server_info` y se conectó al puerto TCP anunciado.

- Se probaron movimiento, bandera, victoria y nueva ronda sin reconexión.

Evidencia asociada: tests/evidence/prueba_vpn_cliente1.png y prueba_vpn_cliente2.png

Commit de referencia: feat: adaptar red y reglas al protocolo actualizado

Hash del commit: 65e2f47 Fecha: 28 de julio de 2026

Experiencia personal de la versión

Reflexión personal

La prueba por VPN fue importante porque reprodujo una situación más cercana a las pruebas entre compañeros. El descubrimiento unicast permitió mantener el flujo del protocolo aun cuando el broadcast de una VPN no fuera confiable. Esta etapa también evidenció la importancia de validar estrictamente los mensajes para interoperar con proyectos escritos en otros lenguajes.

Versión 7. Interfaz de descubrimiento y conexión

Objetivo: Permitir que el usuario encuentre, seleccione y conecte a un servidor sin escribir argumentos de terminal.

Implementación principal

- Se creó server_browser.tscn como User Interface.
- Se integró la interfaz en un CanvasLayer dentro de GameWorld.
- Se creó server_browser.gd para controlar botones y lista.
- ServerDiscovery entrega los resultados en lugar de seleccionar automáticamente.
- Se agregaron búsqueda automática, búsqueda por IP y TCP directo.
- La interfaz aplica el nombre del jugador y solicita la conexión.
- Al recibir welcome, la interfaz se oculta y el juego captura el mouse.

Pruebas y resultado

- La búsqueda mostró servidores encontrados en el Wi-Fi local.
- El usuario pudo seleccionar una dirección y conectarse.
- Dos clientes iniciaron countdown y completaron una ronda.
- Después de game_over comenzó una nueva ronda sin reconexión.

Commit de referencia: feat: agregar interfaz para descubrir y seleccionar servidores

Hash del commit: b80871e Fecha: 28 de julio de 2026

Experiencia personal de la versión

Reflexión personal

La interfaz convirtió una prueba técnica de terminal en un flujo utilizable. Durante la integración fue necesario corregir rutas de nodos y asignar tamaño al ItemList para visualizar los resultados. La prueba final confirmó que la interfaz no cambia el protocolo: únicamente reemplaza los argumentos manuales por controles visuales.

Versión 8. Visualización multijugador completa y corrección del descubrimiento por VPN

Objetivo: Completar la representación visual de todos los jugadores conectados, tanto en el cliente como en el modo servidor, y corregir un problema real de descubrimiento detectado en pruebas remotas por VPN.

Implementación principal

- Se crearon `remote_player.gd` y `remote_player.tscn` para representar visualmente a los demás jugadores dentro de `game_world.tscn`, con interpolación suave de posición.
- `game_client.gd` crea, mueve y elimina las cápsulas de los jugadores remotos según el contenido del arreglo `players[]` recibido en cada mensaje `state`.
- Se corrigió la posición visual de la bandera portada: antes solo se reparentaba al jugador local; ahora se reparenta a cualquier portador, local o remoto, con un desplazamiento elevado para que sea claramente visible.
- Se agregó una etiqueta `Label3D` con el nombre de cada jugador remoto, tomado del mensaje `lobby` (el único que asocia `id` y `name`).
- Se creó `server_player_view.tscn` y se extendió `game_server.gd` para que el servidor muestre visualmente el mapa, el círculo, la bandera y a todos los jugadores conectados en tiempo real, con cámara cenital fija y un HUD con la fase de la partida, el puerto TCP y la cantidad de jugadores.
- Se corrigió el tamaño de las etiquetas de nombre: se quitó `fixed_size` en `Label3D` para que el texto escale con la distancia de la cámara, tanto en el cliente como en la vista del servidor.
- Se corrigió `server_discovery.gd` para detectar automáticamente todas las direcciones IPv4 locales, incluyendo adaptadores virtuales como el de `ZeroTier`, y calcular el broadcast de cada subred sin depender de un argumento manual; se filtraron además las direcciones `link-local` `169.254.x.x`.

Pruebas y resultado

- Se detectó, en prueba remota con una compañera de clase por `ZeroTier`, un descubrimiento asimétrico: ella encontraba el servidor propio, pero no ocurría en sentido inverso.
- Se determinó que la causa era la falta de broadcast automático de subred, ya que el paquete `discover` solo salía hacia `255.255.255.255`, dirección que normalmente no cruza el adaptador virtual de la VPN.
- Tras implementar la detección automática de subredes, se comprobó el descubrimiento correcto en ambos sentidos y sobre múltiples adaptadores virtuales presentes en la misma máquina, sin argumentos adicionales.
- Se verificó en consola que los intentos de envío hacia direcciones `169.254.x.x` desaparecieron tras el filtro.

Evidencia asociada: `tests/evidence/descubrimiento_udp_prueba.png`

Commit de referencia: `feat: agregar visualización de jugadores remotos, vista espectadora del servidor y corregir descubrimiento UDP por subred`

Hash del commit: `c8c2f9a` Fecha: 28 de julio de 2026

Experiencia personal de la versión

Reflexión personal

Esta versión cerró la brecha más visible del proyecto: hasta entonces cada cliente solo veía su propio movimiento. Instanciar y sincronizar las cápsulas remotas a partir de `state`, sin tocar el protocolo, confirmó que el diseño servidor-autoritativo ya contenía toda la información necesaria para la representación visual completa. La corrección del descubrimiento por VPN fue igual de importante: un

error real detectado junto a una compañera de clase, no un caso hipotético, y su solución (detectar subredes automáticamente) es la que finalmente hace posible que el proyecto interopere con otros equipos sin coordinación manual de direcciones.

Versión 9. Conteo regresivo visual, menú de pausa y filtro de servidores propios

Objetivo: Mejorar la usabilidad final del cliente: mostrar el conteo regresivo en pantalla, permitir pausar la partida y depurar la lista de servidores encontrados.

Implementación principal

- Se agregó un Label centrado (UI/CountdownLabel dentro de game_world.tscn) que muestra el número del conteo regresivo (5 a 1) recibido en countdown, oculto automáticamente al iniciar la partida, volver al lobby, terminar la ronda o perder la conexión.
- Se creó un menú de pausa (scenes/ui/pause_menu.tscn y scripts/ui/pause_menu.gd) que se abre con ESC durante la partida, libera el mouse y ofrece Reanudar (cierra el menú y recaptura el mouse sin afectar la conexión) y Salir (cierra el programa por completo).
- Se corrigió server_discovery.gd para que register_server() descarte cualquier respuesta cuya IP de origen coincida con una de las direcciones IPv4 locales de la propia máquina, evitando servidores propios duplicados por cada interfaz de red activa.

Decisión de alcance

La primera versión del botón de pausa incluía la opción "Volver al menú", que desconectaba del servidor y regresaba al buscador para unirse a otra partida sin cerrar el programa. Se detectaron errores derivados de ese flujo, como la desaparición de la bandera en la siguiente partida tras reconectar. Por falta de tiempo para corregir la causa raíz antes de la entrega, se simplificó la función del botón a Salir, que cierra el programa por completo y evita el estado inconsistente.

Pruebas y resultado

- Se comprobó que el conteo regresivo se muestra en pantalla y se oculta correctamente en cada transición de fase.
- Se comprobó que ESC abre el menú de pausa sin perder la conexión y que Reanudar recaptura el mouse correctamente.
- Se comprobó que, con varias interfaces de red activas en la misma máquina, el servidor propio deja de aparecer duplicado en la lista de búsqueda.

Commit de referencia: feat: agregar conteo regresivo visual y menú de pausa con opción de salir

Hash del commit: cf3af61 Fecha: 28 de julio de 2026

Experiencia personal de la versión

Reflexión personal

Esta última etapa no modificó el protocolo ni las reglas del juego: fue exclusivamente presentación y usabilidad. Aun así, resultó relevante porque expuso un límite de tiempo real frente a un bug de reconexión, y la decisión de simplificar el botón de salida en lugar de perseguir una solución completa fue, en retrospectiva, la decisión correcta para llegar a la entrega con un flujo estable en lugar de uno más ambicioso pero frágil.

10. Pruebas realizadas y evidencia

10.1 Estrategia de pruebas

Las pruebas se realizaron de forma incremental. Primero se validaron componentes locales, luego comunicación en una sola computadora, después dos clientes y finalmente comunicación remota por VPN. Este orden permitió aislar problemas de lógica, serialización, red y firewall.

Prueba	Entorno	Resultado
Greybox y colisión	Godot local	Escenario visible y suelo con colisión.
Movimiento y cámara	Cliente local	WASD, mouse, gravedad y límites.
Reglas locales	Cliente local	Captura y victoria provisional.
TCP con un cliente	127.0.0.1	join, welcome, countdown, start y state.
Dos clientes	Misma PC	Countdown y partida coordinada.
Descubrimiento broadcast	Wi-Fi doméstico	Servidor visible en lista; se observaron varias interfaces de red.
Descubrimiento unicast VPN	Dos computadoras / ZeroTier	server_info y conexión TCP correctos.
Ronda completa	Dos clientes	Movimiento, bandera, game_over y regreso al lobby.
Interfaz	Cliente sin flags	Búsqueda, selección, conexión y ocultamiento correctos.
Jugadores remotos	Dos clientes, misma PC	Cápsulas remotas visibles, con nombre y bandera reparentada correctamente.
Vista del servidor	Proceso servidor	Mapa, círculo, bandera y jugadores visibles con cámara cenital y HUD.
Descubrimiento VPN bidireccional	Dos computadoras, ZeroTier	Ambos sentidos encuentran el servidor tras detectar subredes automáticamente.
Conteo, pausa y filtro propio	Cliente local	Countdown en pantalla, ESC pausa sin perder conexión, servidor propio no duplicado.

10.2 Evidencia esperada en el repositorio

- greybox_v0.2.0.png
- prototipo_funcional_v0.3.0.png
- prueba_conexion_tcp_1.png
- prueba_dos_conexiones.png
- prueba_vpn_cliente1.png
- prueba_vpn_cliente2.png
- descubrimiento_udp_prueba.png
- Captura de la interfaz con servidores encontrados.
- Captura de dos clientes durante una partida.

Recomendación para la entrega

Agregar debajo de cada evidencia una fecha, breve descripción, versión y commit asociado. También conviene conservar capturas del servidor y de los clientes mostrando los mensajes relevantes de consola.

11. Uso de Git e inteligencia artificial

11.1 Git como cronología

El proyecto utiliza commits funcionales para relacionar el código con la documentación. La bitácora registra objetivo, cambios, pruebas, resultado, limitaciones y evidencia de cada versión. Los hashes y fechas de cada commit, obtenidos desde git log, se incluyen en la tabla siguiente y junto a cada versión en la sección 9.

Versión	Mensaje de commit
1	chore: configurar estructura inicial del proyecto Godot
2	feat: crear primera versión visual 3D del escenario
3	feat: implementar jugador y reglas locales de prueba
4	feat: implementar primera conexión TCP cliente-servidor
5	feat: implementar descubrimiento UDP y conexión de dos clientes
6	feat: adaptar red y reglas al protocolo actualizado
7	feat: agregar interfaz para descubrir y seleccionar servidores
8	feat: agregar visualización de jugadores remotos, vista espectadora del servidor y corregir descubrimiento UDP por subred
9	feat: agregar conteo regresivo visual y menú de pausa con opción de salir

11.2 Uso de inteligencia artificial

Se utilizó ChatGPT (GPT-5.6 Sol, nivel de inteligencia media) como apoyo para aprender GDScript, proponer estructuras iniciales de scripts, interpretar requisitos del protocolo, diagnosticar errores y preparar documentación. A partir de la versión 8 se cambió a Claude Sonnet 5 (nivel de inteligencia media), debido a errores técnicos que impidieron seguir utilizando la versión gratuita de ChatGPT. En ambos casos, el código generado se utilizó como base de estudio y fue probado, ajustado e integrado por etapas.

- Generación inicial del movimiento y cámara del jugador.
- Primera estructura cliente-servidor TCP.
- Descubrimiento UDP y conexión de dos clientes.
- Adaptación al protocolo actualizado.
- Integración de la interfaz de selección de servidores.
- Redacción y organización de bitácora y documentación.

- Visualización de jugadores remotos, vista gráfica del servidor y corrección del descubrimiento UDP por subred (Claude Sonnet 5).
- Conteo regresivo visual, menú de pausa y filtro de servidores propios (Claude Sonnet 5).

El registro literal de prompts, archivos relacionados y modificaciones realizadas se encuentra en docs/prompts_ia.md. Las capturas de los prompts se almacenan en docs/prompts_ia/. Esta separación evita sobrecargar el cuerpo principal, pero mantiene la trazabilidad exigida por el proyecto.

12. Ejecución del proyecto

12.1 Servidor

```
& "C:\ruta\Godot_v4.7-stable_win64_console.exe" --path . "scenes/network/server.tscn"
```

El servidor abre TCP 8889 y UDP 8888. Debe permanecer en ejecución mientras los clientes buscan y juegan.

12.2 Cliente con interfaz

```
& "C:\ruta\Godot_v4.7-stable_win64_console.exe" --path . "scenes/levels/game_world.tscn"
```

El cliente muestra ServerBrowser. El usuario escribe su nombre, busca servidores, selecciona uno y presiona Conectar.

12.3 Prueba manual de respaldo

- Buscar por IP: escribir la IP del servidor y enviar discover UDP unicast.
- Conexión directa: escribir IP y puerto TCP conocido.
- En ZeroTier: utilizar la IP virtual administrada del equipo servidor.

12.4 Firewall

El equipo servidor debe permitir tráfico entrante UDP 8888 y TCP 8889 en el perfil de red utilizado. En una VPN, también deben verificarse las reglas del adaptador virtual y la autorización de ambos dispositivos.

13. Estado actual, limitaciones y trabajo pendiente

13.1 Funciones completadas

- Escenario 3D y jugador en primera persona.
- Cliente y servidor mediante sockets básicos.
- Descubrimiento UDP y conexión TCP.
- Framing, buffers y JSON UTF-8.
- Lobby, countdown, partida, game_over y regreso al lobby.
- Spawn, movimiento, captura, robo, bandera y victoria autoritativos.
- Prueba local con dos clientes.
- Prueba remota mediante VPN.
- Interfaz para buscar, seleccionar y conectar.
- Representación visual de todos los jugadores remotos y de la bandera portada, dentro de cada cliente.
- Vista gráfica del modo servidor con mapa, jugadores y HUD.

- Descubrimiento UDP corregido para redes VPN con múltiples interfaces y subredes.
- Conteo regresivo visible en pantalla, menú de pausa y filtro de servidores propios en la búsqueda.

13.2 Limitaciones actuales

- La capacidad de 100 conexiones está configurada, pero no ha sido sometida a una prueba de carga con esa cantidad.
- La interoperabilidad se ha comprobado entre instancias de este proyecto y con una compañera de clase por VPN, pero no todavía con todos los proyectos heterogéneos de la clase.
- El cálculo automático del broadcast de subred asume máscara /24; en redes con una máscara distinta seguiría siendo necesario el respaldo manual por IP.
- La orientación de los jugadores remotos no se replica: todos se muestran con una orientación fija, ya que el protocolo acordado por la clase no contempla ese campo.
- El menú de pausa no ofrece una forma de volver al buscador de servidores sin cerrar el programa; hacerlo de forma segura requeriría corregir un bug de la bandera al reconectar, fuera del alcance de esta entrega por tiempo.
- El filtro de servidores propios en la búsqueda es una heurística basada en las IPs locales de la máquina, no una garantía del protocolo.
- Falta una presentación visual de errores y desconexión del servidor dentro de la UI del cliente.

13.3 Próximos pasos recomendados

13. Completar la prueba de carga con un número de conexiones cercano al límite de 100.
14. Coordinar y ejecutar la prueba de interoperabilidad formal con los proyectos de toda la clase.
15. Agregar manejo visual de errores y de desconexión del servidor dentro de la UI del cliente.
16. Investigar y corregir el error de la bandera al reconectar, para poder reincorporar de forma segura la opción de volver al buscador de servidores desde el menú de pausa.
17. Evaluar, en una futura iteración fuera de este curso, un campo opcional de orientación acordado con la clase, si se decidiera ampliar el protocolo.

14. Conclusiones

El desarrollo por versiones permitió transformar una idea visual local en una aplicación multijugador completa con protocolo definido. La separación entre servidor y cliente, el framing de TCP y la autoridad del servidor constituyeron los aprendizajes técnicos principales. La implementación final demuestra descubrimiento, conexión, sincronización visual completa de todos los jugadores y ciclo de partida, tanto en una computadora como a través de una VPN.

La versión 7 eliminó la necesidad de especificar argumentos de conexión en la terminal; las versiones 8 y 9 completaron los requisitos visuales pendientes del entregable: todos los jugadores conectados ahora se ven entre sí en cada cliente, el modo servidor muestra gráficamente la partida completa, el descubrimiento por VPN funciona en ambos sentidos sin argumentos manuales, y el conteo regresivo y el menú de pausa mejoran la experiencia final. Lo que queda abierto para después de la entrega individual es la prueba de interoperabilidad formal con los proyectos heterogéneos de toda la clase y una prueba de carga con un número de conexiones cercano al límite de 100.

La bitácora, los commits, las evidencias y el registro de prompts proporcionan trazabilidad completa del proceso desde la versión 1 hasta la versión 9. Esta documentación corresponde al estado final entregado en el repositorio.

Anexo A. Lista de verificación previa a la entrega

- ☑ El repositorio abre correctamente en Godot 4.7 Stable.
- ☑ Existe un commit identificable por cada una de las 9 versiones documentadas.
- ☑ Los hashes y fechas están completos (ver secciones 9 y 11.1).
- ☑ La carpeta tests/evidence contiene todas las capturas citadas.
- ☑ El servidor escucha UDP 8888 y anuncia su puerto TCP.
- ☑ El cliente muestra servidores y permite selección.
- ☑ Existe respaldo por IP y por IP:puerto.
- ☑ Los nombres y mensajes cumplen límites y formato.
- ☑ Dos o más clientes ven a todos los jugadores (desde la versión 8).
- ☑ La vista del servidor muestra el estado de todos los jugadores (desde la versión 8).
- ☑ Captura, robo, victoria y desconexión del portador funcionan.
- ☑ El ciclo vuelve al lobby sin reconexión.
- ☑ Se probó con proyectos de otros compañeros tanto con VPN y Hostspots
- ☑ Se registró la experiencia personal de cada versión.
- ☑ prompts_ia.md contiene los prompts y archivos asociados.

Anexo B. Registro final de identificación

Dato	Valor final
URL del repositorio	https://github.com/RaquelU/Project1_CC8_2026
Rama entregada	main
Commit final	cf3af61
Versión final del proyecto	Versión 9 (entrega final)
Versión del protocolo	1
Versión del documento de protocolo	1.2.0
Fecha de entrega	28 de julio de 2026

Anexo C. Fuentes normativas y documentos relacionados

- Proyecto 1 - Captura la Bandera Multijugador. Documento de instrucciones, reglas generales, limitaciones y entregables del curso.
- Estándar Protocolo CTF - CC8 2026. Especificación técnica independiente, documento 1.2.0 y protocolo de cable v1.
- docs/01_bitacora.md. Cronología de implementación.

- docs/prompts_ia.md. Registro de uso de inteligencia artificial.
- Repositorio Project1_CC8_2026 y su historial de Git.